

Prédiction et Optimisation intelligente et distribuée de ressources dans le Network Slicing 5G

Nahil EL BEZZARI
Yassine LAZIZI
Nadir NEHILI

CY Cergy Paris Université
CY Tech école d'ingénieurs
Cergy, France



29 mars 2026

Résumé

L'avènement des réseaux 5G et de la future 6G repose fondamentalement sur le paradigme du découpage du réseau (Network Slicing), permettant de déployer simultanément de multiples services aux exigences hétérogènes sur une infrastructure physique partagée. Cependant, la gestion dynamique de l'allocation des ressources radio face aux fluctuations du trafic constitue un défi majeur. Si les solutions historiques se sont avérées majoritairement réactives, se contentant de corriger les dégradations a posteriori, la littérature récente s'oriente vers des architectures proactives propulsées par l'Intelligence Artificielle, couplant l'anticipation du trafic à des moteurs de réallocation préventive.

Dans le cadre de cette étude, nous dressons un état de l'art critique de ces nouvelles architectures prédictives et décisionnelles, en identifiant les verrous ouverts et les axes de recherche les plus prometteurs.

1 État de l’art

Les anciennes solutions de gestion dynamique de slices étaient souvent réactives ou périodiques, conduisant à un ajustement de l’allocation des ressources en réponse à une dégradation constatée. Ces approches souffrent donc d’un défaut structurel car elles ne peuvent pas empêcher les violations des Service Level Agreement (SLA) mais simplement les corriger après coup.

Ce constat pousse donc vers une démarche proactive où l’on cherche à prédire le trafic afin d’allouer les ressources nécessaires en amont de la violation. En ce sens, la littérature actuelle converge vers des architectures en deux blocs :

- **Un prédicteur** estime le trafic ou les métriques futures à un horizon donné.
- **Un moteur de réallocation** ajuste les ressources en réponse au risque de violation détecté.

1.1 Modèle de prédiction

Un premier axe de différenciation concerne l’architecture du modèle de prédiction elle-même. [13] propose *ConvLSTMTransNet*, un pipeline hybride en trois étages. Le premier étage est constitué de couches convolutives 1D qui appliquent des filtres glissants sur la série temporelle d’entrée afin d’extraire des motifs locaux (pics de trafic, variations périodiques à court terme). Les feature maps résultantes sont ensuite injectées dans un réseau LSTM dont les cellules à portes (forget, input, output) permettent de capturer les dépendances temporelles à moyen terme en régulant sélectivement le flux d’information à travers la séquence. Enfin, la sortie du LSTM alimente la section encodeur d’un Transformer, où le mécanisme de multi-head self-attention calcule des scores d’attention entre toutes les positions temporelles simultanément, permettant au modèle de pondérer des dépendances éloignées que le LSTM seul peine à maintenir. Le modèle est évalué sur des données de télémétrie SNMP issues de routeurs de périphérie, avec des pas de prédiction variant de 5 minutes à plusieurs heures. Il surpasse les baselines (RNN, GRU, LSTM simple) en termes de RMSE et de MAE, particulièrement sur les horizons moyens à élevés où l’attention multi-tête apporte le gain le plus significatif. Toutefois, le modèle se limite à l’historique univarié du trafic comme unique entrée et ne prend en compte ni l’état courant du réseau (charge CPU, utilisation mémoire des nœuds, taux d’occupation des liens) ni la nature des services transportés, ce qui réduit sa capacité à anticiper des dégradations spécifiques à une tranche.

Cette question de l’horizon optimal se retrouve chez Tuna et al. [16], qui mènent une comparaison systématique de quatre architectures (LSTM, CNN 1D, Transformer, LightGBM) sur un jeu de données opérateur couvrant des centaines de stations de base. Leur protocole évalue chaque modèle sur des horizons allant de 1 pas (1 heure) à 24 pas, en mesurant à la fois l’erreur brute et le taux de violation SLA résultant. Sur des horizons courts (1 à 4 heures), le modèle LSTM, alimenté par des features multivariées construites par expertise métier (coordonnées spatiales, indicateur d’heure de pointe, jour de la semaine), surpasse le Transformer dont le mécanisme d’attention ne dispose pas de suffisamment de contexte futur pour s’exprimer. Sur des horizons plus lointains (8 à 24 heures), la capacité du Transformer à capturer des corrélations globales via l’attention lui confère un avantage décisif. Cette dualité soulève une question pratique : un système opérationnel devrait-il maintenir deux modèles distincts selon l’horizon visé, au prix d’une complexité de déploiement et de maintenance accrue ?

Z. Min et al. [9] adoptent une architecture purement Transformer avec leur modèle *TST* (Time Series Transformer). Celui-ci utilise un encodage positionnel sinusoïdal pour injecter l’information d’ordre temporel dans les embeddings d’entrée, suivi de plusieurs couches d’encodeur Transformer empilées. La particularité de leur approche réside dans le pré-traitement : un lissage exponentiel double de Holt est appliqué en amont pour décomposer la série en une composante de niveau et une composante de tendance, éliminant ainsi le bruit haute fréquence du réseau avant que le Transformer ne traite la série lissée. Sur des jeux de données réels de trafic 5G, le modèle combiné (Holt + TST) réduit le MAPE de 15 à 30% par rapport aux architectures classiques (GRU, LSTM) et aux modèles hybrides antérieurs (SES+LSTM), avec un avantage particulièrement marqué sur les prédictions à long terme allant jusqu’à un mois. Au-delà du modèle de prédiction, les auteurs l’intègrent au sein d’un Jumeau Numérique (Digital Twin) du réseau 5G. Ce jumeau réplique en temps réel l’état du réseau physique et fonctionne selon une boucle proactive *MAPE* (Monitor, Analyze, Plan and Execute) : la phase *Monitor* collecte les métriques réseau, *Analyze* déclenche la prédiction via le TST, *Plan* génère un plan de réallocation des ressources, et *Execute* applique les décisions sur l’infrastructure via des API de contrôle. Si cette approche systémique est la plus complète du corpus étudié, elle introduit une dépendance forte à la fidélité du jumeau numérique : tout écart de modélisation entre le modèle virtuel et l’infrastructure réelle (latences non modélisées, comportements de file d’attente simplifiés) se propage directement dans les décisions d’allocation.

Indépendamment du choix architectural, un second axe concerne la nature des données fournies au modèle. La plupart des approches, y compris *ConvLSTMTransNet*, se limitent à l'historique univarié du trafic. *LSTM-FSD* [15] adopte une perspective fondamentalement différente en construisant un vecteur d'entrée multivarié qui encode l'état global du réseau sur une fenêtre glissante de 5 minutes. Ce vecteur comprend : le type de tranche (*slice type*), le débit agrégé par lien, le taux d'utilisation des ressources radio (PRB usage) et la charge de capacité résiduelle des liens. L'architecture LSTM-FSD (LSTM with Full State Description) traite cette séquence multivariée pour produire une prédiction du volume de trafic par tranche à l'horizon suivant. La véritable originalité réside cependant dans le module *LP-KPI* (Linear Programming KPI), un solveur analytique post-prédiction qui, à partir du trafic prédit et de la topologie courante du réseau, résout un système d'équations de files d'attente pour estimer la latence de bout en bout et le taux de perte de paquets par flux. Ce découplage prédiction/estimation permet un temps de calcul de seulement 80 μ s par évaluation KPI.

Leur modèle a obtenu un MAPE de 18,66% sur des données de test inédites, et *LP-KPI* a démontré une pénalité de délai de 0.153 (écart moyen normalisé entre latence prédite et mesurée). Cependant, un MAPE de 18,66% reste élevé pour une fenêtre de seulement 5 minutes : concrètement, cela signifie qu'en moyenne, le trafic prédit s'écarte de près d'un cinquième du trafic réel, ce qui peut entraîner un sous-dimensionnement ou un sur-dimensionnement significatif des ressources. Dans un contexte de réallocation proactive, une erreur de cette ampleur pourrait elle-même provoquer des violations de SLA plutôt que les prévenir, particulièrement pour les tranches URLLC dont les marges de tolérance sont extrêmement faibles.

Tuna et al. [16], en plus de leur comparaison architecturale, enrichissent également les entrées en ajoutant des features construites par expertise métier : coordonnées géographiques de la station de base (latitude, longitude), indicateurs binaires d'heure de pointe et encodage cyclique du jour de la semaine. Cette convergence entre les deux travaux confirme l'importance d'aller au-delà du simple historique temporel pour intégrer le contexte spatial et opérationnel, sans qu'un consensus émerge cependant sur l'ensemble optimal de features à considérer ni sur la méthode de sélection de ces features.

Un troisième axe, plus rarement abordé, concerne l'adéquation de la fonction de perte aux besoins métier. Les fonctions standards telles que la MSE ($\frac{1}{n} \sum (y_i - \hat{y}_i)^2$) ou la MAE ($\frac{1}{n} \sum |y_i - \hat{y}_i|$) sont symétriques : elles pénalisent identiquement une surestimation de $+k$ et une sous-estimation de $-k$. Or, dans un contexte de *network slicing*, ces deux erreurs n'ont pas le même coût opérationnel : une sous-estimation du trafic conduit à un sous-provisionnement et donc à une violation de SLA, tandis qu'une surestimation n'entraîne qu'un léger gaspillage de ressources.

Tuna et al. [16] abordent directement ce problème. Ils introduisent deux fonctions de perte paramétriques : la *wMAE* (weighted MAE) et la *wMSE* (weighted MSE), qui appliquent un coefficient de pénalité $\alpha > 1$ lorsque la prédiction est inférieure à la valeur réelle (sous-estimation) et un coefficient $\beta < 1$ lorsqu'elle est supérieure (surestimation). Formellement :

$$\text{wMAE} = \frac{1}{n} \sum_{i=1}^n w_i \cdot |y_i - \hat{y}_i| \quad \text{où } w_i = \begin{cases} \alpha & \text{si } \hat{y}_i < y_i \\ \beta & \text{sinon} \end{cases} \quad (1)$$

Le ratio α/β est calibré pour atteindre un taux de violation SLA cible fixé par l'opérateur. Cette idée, bien que simple dans sa formulation, a des implications profondes car elle aligne directement la fonction objectif de l'entraînement sur la contrainte opérationnelle de respect des SLA. Les auteurs montrent expérimentalement qu'à erreur globale comparable, l'utilisation de la *wMAE* réduit significativement le nombre de violations SLA par rapport à une MAE standard. Il est notable qu'aucun des autres travaux étudiés n'intègre ce type de biais asymétrique, ce qui constitue une lacune récurrente de la littérature.

Les approches précédentes reposent toutes sur un entraînement centralisé où l'ensemble des données de toutes les stations est agrégé sur un serveur unique pour entraîner un modèle global. Ce paradigme pose un double problème à l'échelle d'un réseau déployé : d'une part, la centralisation des données brutes de trafic engendre un volume de communication et une consommation énergétique élevés ; d'autre part, un modèle unique tend à moyenniser les comportements hétérogènes des différentes stations, dégradant la qualité des prédictions locales.

N. Pavlidis et al. [11] abordent cette problématique via l'apprentissage fédéré selon le protocole *FedAvg*. Chaque station de base entraîne localement un modèle LSTM sur ses propres données de trafic pendant un nombre fixe d'époques locales, puis transmet uniquement les gradients agrégés (mises à jour de poids) à un serveur central. Celui-ci calcule la moyenne pondérée des mises à jour reçues et redistribue le modèle global mis à jour. Ce cycle est répété sur plusieurs rounds de communication. Le modèle est évalué sur un jeu de données réel issu de 28 stations de base situées à Barcelone, couvrant des

environnements variés (gare, zone résidentielle, stade, centre commercial). Les résultats montrent que l’approche fédérée capture mieux la granularité locale que le modèle centralisé : les prédictions sont plus précises pour les stations présentant des profils de trafic atypiques (pics sportifs au stade, flux pendulaires en gare).

Les auteurs introduisent un indicateur de soutenabilité S défini comme le produit normalisé de trois facteurs : l’erreur de prédiction (MAPE), l’énergie totale consommée par l’entraînement et le volume de données échangé entre les stations et le serveur. Un S faible indique un bon compromis. L’apprentissage fédéré obtient $S = 30.24$ contre $S = 132.03$ pour l’approche centralisée, soit une amélioration d’un facteur 4,4. Néanmoins, plusieurs limites méritent d’être soulignées. L’évaluation est conduite exclusivement sur des données barcelonaises, ce qui ne permet pas de conclure quant à la généralisabilité de l’approche à des contextes urbains différents (densité de population, mix de services, topographie réseau). Les auteurs n’abordent pas non plus la robustesse du modèle face à des distributions de trafic non-stationnaires ou à des événements exceptionnels (concerts, catastrophes), scénarios où les données locales deviennent soudainement non représentatives des patterns appris.

Un dernier axe concerne la représentation spatiale du réseau lui-même. Les modèles à base de CNN, y compris *ConvLSTMTransNet* [13], forcent les données dans une structure en grille régulière à la manière des pixels d’une image : les stations de base sont projetées sur une matrice 2D selon leurs coordonnées géographiques, et les convolutions spatiales opèrent sur cette grille. Or, les réseaux de télécommunications ne sont pas des grilles régulières mais des graphes complexes où la connectivité entre nœuds dépend de la topologie physique (fibre, faisceaux hertziens) et logique (tunnels MPLS, VPN), et où les liens présentent des caractéristiques hétérogènes (capacité, latence, taux de perte).

L’architecture classique pour répondre à ce besoin est *ST-GCN* (Spatial-Temporal Graph Convolutional Networks), qui remplace les convolutions 2D par des convolutions sur graphe : au lieu de filtrer un voisinage spatial régulier, chaque nœud agrège les informations de ses voisins selon la matrice d’adjacence du graphe réseau. Cependant, [18] identifie une limite supplémentaire : les GCN standards utilisent une unique représentation du graphe (typiquement la matrice d’adjacence normalisée), ce qui ne capture pas l’hétérogénéité des liens entre nœuds ni les différents modes de propagation du trafic.

Les auteurs introduisent donc *MRSTCGN* (Multi-Representation Spatial-temporal Graph Convolutional Network). Le principe est de construire K matrices de représentation du même graphe, chacune capturant un aspect différent de la topologie : la matrice d’adjacence directe (connectivité physique), la matrice de distance géodésique (proximité topologique), la matrice de corrélation de trafic (similarité statistique des séries temporelles entre nœuds), etc. Pour chaque représentation k , une branche GCN indépendante extrait des embeddings spatiaux, et un module d’attention apprend à pondérer dynamiquement la contribution de chaque branche selon le contexte temporel. Les embeddings fusionnés sont ensuite traités par un module temporel (GRU ou Temporal Convolution) pour produire la prédiction finale.

Les expériences, menées sur deux jeux de données publics de trafic réseau (PeMSD4 et PeMSD8, comportant respectivement 307 et 170 capteurs), démontrent que *MRSTCGN* surpasse les baselines *ST-GCN*, *ASTGCN* et *Graph WaveNet* en termes de MAE, RMSE et MAPE. Cette avancée reste cependant évaluée sur des données de trafic routier réutilisées comme proxy pour le trafic télécom, sans segmentation par type de tranche ni par type de service, ce qui laisse ouverte la question de son applicabilité directe au *network slicing* où les corrélations spatiales entre tranches d’un même nœud pourraient différer significativement.

L’analyse de ces six travaux selon les axes identifiés fait émerger plusieurs constats. Premièrement, chaque contribution se concentre sur un aspect spécifique du problème (architecture, entrées, fonction de perte, paradigme d’entraînement, modélisation spatiale) sans qu’aucune ne propose une approche les unifiant : aucun modèle ne combine simultanément l’enrichissement multivarié des entrées, des fonctions de perte asymétriques sensibles aux SLA, un entraînement distribué et une modélisation spatiale par graphes. Deuxièmement, la plupart des évaluations sont conduites sur des jeux de données soit purement simulés, soit géographiquement limités à une seule ville, soit issus d’un domaine connexe (trafic routier), ce qui fragilise les conclusions en termes de généralisation au trafic télécom réel multi-tranche. Troisièmement, la question du passage de la prédiction à la décision d’allocation reste largement traitée de manière découplée, à l’exception notable du jumeau numérique de [9], dont la complexité de mise en œuvre et le coût de maintenance du modèle virtuel peuvent constituer un frein à l’adoption opérationnelle. Enfin, l’intégration des contraintes SLA dès la phase d’entraînement, pourtant essentielle pour un usage proactif, n’est abordée que par un seul travail [16], laissant un espace significatif pour des approches qui aligneraient nativement l’objectif d’apprentissage sur les exigences de qualité de service.

TABLE 1 – Comparaison des modèles de prédiction de trafic et de performances

Référence	Architecture	Variables d'entrée	Horizon de prédiction	Métriques clés	Contribution principale
[13]	ConvLSTMTransNet	Historique de trafic (télémétrie)	Très court : 5min	MAE 0,30, RMSE 0,48 WAPE 3,27%	Extraction locale (CNN) + dépendances temporelles (LSTM) + long terme (multi-head self-attention).
[15]	LSTM-FSD + LP-KPI	Type de tranche, débit, utilisation des ressources, charge des liens (5 min)	Très court : 5min	MAPE 18,66%	Estimation des KPIs (latence, perte) via l'état global du réseau.
[16]	LSTM (court terme) / Transformer (long terme)	Historique du trafic + caractéristiques spatiales + heures de pointe	Court : 1–4 h Long : 8–24 h	Court : wMAE 0,474 Long : wMAE 0,823	Fonctions de perte asymétriques intégrant la contrainte de violation SLA.
[11]	Apprentissage fédéré (DL distribué)	Données de trafic locales par station	Court : 15min–1 h	NRMSE 1,385	Entraînement distribué réduisant l'énergie et capturant les comportements hétérogènes.
[18]	MRSTCGN (Multi-Rep ST-GCN)	Topologie du réseau (graphe) + trafic	-	MAE : 1,14–3,02 RMSE : 2,36–8,09 MAPE : 4,53%–16,47%	Projection multi-sous-espaces pour capturer l'hétérogénéité des liens.
[9]	Holt Smoothing + TST (Transformer)	Données de trafic réelles + boucle proactive MAPE	Très long : jusqu'à 1 mois	Court : RMSE 0,0163 MAE 0,0107 MAPE 6,77% Long : MAPE 23,125%	Intégration dans un Jumeau Numérique 5G pour éviter les risques d'allocation directe.

1.2 Modèle d'allocation

1.2.1 Approche mathématique et heuristique

Un premier axe structurant oppose les méthodes qui garantissent l'optimalité mathématique à celles qui la sacrifient au profit de la rapidité d'exécution.

[5] se place du côté de l'optimisation exacte. Les auteurs constatent que les modèles classiques imposent souvent un routage mono-chemin pour réduire la complexité algorithmique, ce qui se fait au détriment de l'efficacité spectrale. Ils formulent le problème d'allocation comme un programme non-linéaire en nombres entiers mixtes (MINLP) intégrant simultanément le routage multi-chemin, les contraintes de délai de bout en bout et les exigences de fiabilité par tranche. La non-linéarité provient des termes de délai dans les files d'attente, où le délai croît de façon hyperbolique avec le taux d'utilisation du lien. Pour rendre le problème traitable, les auteurs procèdent en deux étapes : d'abord une linéarisation par morceaux des fonctions de délai, transformant le MINLP en un programme linéaire en nombres entiers mixtes (MILP), puis une relaxation continue des variables entières, rendant le problème soluble en temps polynomial. Le MILP relaxé est ensuite résolu via un algorithme de Génération de Colonnes personnalisée (cCG). Dans ce schéma, le *problème maître restreint* sélectionne la combinaison optimale parmi un ensemble de motifs de routage candidats, tandis que le *sous-problème de pricing* génère de nouveaux motifs en résolvant un problème de plus court chemin avec contraintes de ressources (SPPRC) pour chaque demande de service. L'algorithme itère entre ces deux niveaux jusqu'à convergence. Les résultats, obtenus sur une topologie réaliste de 112 nœuds et 440 liens, montrent une résolution quasi-optimale (gap < 1% par rapport à la borne inférieure LP) en moins de 15 secondes, là où le solveur MINLP direct (BARON/SCIP) échoue par dépassement de mémoire ou nécessite plus de 1000 secondes. Cependant, cette approche reste fondamentalement statique, elle calcule une allocation optimale pour un instantané de la demande, sans mécanisme d'adaptation en ligne face aux fluctuations du trafic.

À l'opposé, S. Urooj et al. [17] abandonnent la garantie d'optimalité au profit de la vitesse d'exécution via une approche méta-heuristique. Le contexte est celui des réseaux hétérogènes (HetNets) denses de la 5G, où le partage du spectre radio entre utilisateurs primaires (PU) et secondaires (SU) génère des interférences massives. Les auteurs introduisent l'algorithme hybride *IAVGOA*, qui fusionne deux heuristiques bio-inspirées complémentaires. La composante AVOA (African Vultures Optimization Algorithm) assure l'*exploration* en simulant le comportement de recherche de nourriture des vautours permettant de parcourir largement l'espace des solutions via des sauts aléatoires de grande amplitude, évitant ainsi les minima locaux. La composante GOA (Grasshopper Optimization Algorithm) assure l'*exploitation* en simulant le comportement des sauterelles qui resserrent progressivement la recherche autour des meilleures solutions identifiées, avec des déplacements de faible amplitude guidés par les forces d'attraction et de répulsion entre individus de la population. L'hybridation alterne dynamiquement entre ces deux phases selon un paramètre de température décroissant au fil des itérations. La fonction de fitness évalue simultanément quatre objectifs que sont le coût de communication (proportionnel à la puissance d'émission et au nombre de sauts), le niveau d'interférence agrégée sur les PU, le respect des seuils de QoS (débit minimum, latence maximum) et la consommation énergétique totale. Au-delà de l'allocation fréquentielle, le système sélectionne aussi dynamiquement la topologie réseau la plus adaptée (maillé, étoile, anneau) parmi un ensemble de candidats, en évaluant chaque topologie comme une dimension supplémentaire de l'espace de recherche. Les résultats montrent une réduction du temps de calcul jusqu'à 86% et une efficacité énergétique améliorée de 25 à 60% par rapport aux algorithmes PSO et GA classiques. Toutefois, comme toute méta-heuristique, IAVGOA ne fournit aucune garantie de convergence vers l'optimum global, et la qualité de la solution dépend fortement du réglage des hyperparamètres (taille de population, nombre d'itérations, paramètre de température).

Un second axe concerne la transparence du processus décisionnel. Les approches basées sur le Deep Learning ou le Reinforcement Learning, bien que performantes, agissent comme des boîtes noires dont les décisions sont difficilement auditables par un opérateur réseau. Ce constat motive des approches à base de règles explicites.

[2] propose le framework *CARO* (Context-Aware Resource Orchestration), structuré en trois modules interagissant en boucle fermée selon une architecture SDN/NFV. Le premier module, l'*orchestrateur de tranches*, maintient un vecteur de poids de priorité $\mathbf{w} = [w_{eMBB}, w_{URLLC}, w_{mMTC}]$ qu'il ajuste dynamiquement à chaque cycle de contrôle. L'ajustement suit une règle proportionnelle, lorsque les métriques QoS d'une tranche s se dégradent (par exemple, la latence URLLC dépasse un seuil d'alerte fixé à 80% du SLA), le poids w_s est incrémenté d'un facteur multiplicatif, tandis qu'un mécanisme de normalisation garantit que $\sum_s w_s = 1$, évitant ainsi l'affaiblissement des autres tranches. Le second module, le *contrôleur MEC*, résout un problème de placement de fonctions réseau virtualisées (VNF) sur les

serveurs Edge. Pour chaque VNF à placer, il évalue une fonction de coût multicritère sont personnalisés selon le type de tranche. Le troisième module, le *contrôleur SDN*, calcule le chemin réseau via des algorithmes de plus court chemin adaptés : Dijkstra pondéré par la latence pour les flux URLLC, et par la bande passante résiduelle pour les flux eMBB. Un mécanisme de lissage exponentiel $\hat{L}(t) = \alpha L(t) + (1 - \alpha)\hat{L}(t - 1)$ est ensuite appliqué sur la charge des serveurs pour anticiper les tendances de congestion et imposer des pénalités préventives aux serveurs dont la charge lissée dépasse 70%. Les simulations, conduites sur une topologie MEC à 5 serveurs Edge et 20 stations de base, montrent une latence URLLC maintenue entre 0.32 et 0.39 ms (contre un SLA de 1 ms), et un coefficient de variation de charge inter-serveurs inférieur à 5%, attestant d'un équilibrage quasi-parfait. La force de CARO réside dans son interprétabilité totale, en effet, chaque décision peut être tracée jusqu'à une règle explicite. Sa limite réside cependant dans l'absence d'apprentissage car les seuils, coefficients et règles sont fixés par l'opérateur et ne s'adaptent pas automatiquement à des contextes non prévus.

Les approches précédentes partagent une limitation commune car elles opèrent soit de manière statique, soit selon des règles prédéfinies (CARO). Or, les réseaux 5G/6G sont soumis à des variations rapides et imprévisibles de la demande, ce qui peut nécessiter une adaptation continue.

K. Venkata Ramana et al. [8] abordent cette problématique avec *EvoNetSlice*, un framework qui couple des algorithmes évolutionnaires à un module d'analyse par intelligence artificielle. Le cycle d'optimisation fonctionne comme suit : à chaque intervalle de décision, le module IA extrait des indicateurs de tendance à partir des séries temporelles de trafic et du comportement utilisateur. Ces indicateurs alimentent l'initialisation de la population de l'algorithme évolutionnaire, orientant la recherche vers des régions prometteuses de l'espace des solutions plutôt que de partir d'une population aléatoire. Chaque individu de la population encode une configuration d'allocation (nombre de RBs par tranche, puissance d'émission, paramètres de scheduling). La fonction de fitness est définie comme une combinaison pondérée multi-objectifs :

$$f(R) = \lambda_1 \cdot \text{Throughput}(R) - \lambda_2 \cdot \text{Latency}(R) - \lambda_3 \cdot \text{PacketLoss}(R) \quad (2)$$

où les coefficients λ_i sont eux-mêmes ajustés dynamiquement en fonction de l'écart entre les métriques courantes et les seuils SLA. Ainsi, lorsqu'une tranche s'approche de la violation, le poids de la métrique concernée est augmenté, forçant l'algorithme évolutionnaire à prioriser cette dimension. Les opérateurs génétiques génèrent de nouvelles configurations, et un mécanisme d'élitisme préserve les meilleures solutions d'une génération à l'autre. Le processus converge en quelques dizaines de générations vers une allocation de Pareto-optimale. Les évaluations, menées sur des jeux de données synthétiques multivariés simulant des scénarios 6G démontrent une latence moyenne contenue autour de 0.30 ms avec un débit agrégé supérieur de 15% aux approches statiques. Cependant, le recours exclusif à des données synthétiques constitue une limitation majeure car les corrélations temporelles et spatiales d'un trafic réel (effets de foule, mobilité, handovers) ne sont pas capturées, ce qui fragilise la validité externe des résultats.

L'analyse des ces travaux fait émerger un constat clair. Le compromis entre optimalité et temps de calcul reste un axe de tension non résolu. En effet, les méthodes exactes [5] garantissent l'optimalité mais sont statiques, tandis que les méta-heuristiques [17] et les algorithmes évolutionnaires [8] offrent la rapidité sans garantie de qualité de la solution. En ce sens, il ne paraît pas comme envisageable d'apporter une solution à ces deux problèmes de manière simultanée mais d'orienter son choix en fonction de la direction privilégiée par nos travaux.

TABLE 2 – Comparaison des modèles mathématiques et heuristiques pour l'allocation de ressources

Référence	Méthodologie	Objectifs d'Optimisation	Vitesse de calcul	Latence obtenue	Anticipation
[5]	Optimisation mathématique (MILP + cCG)	Délai E2E, Fiabilité, Coût d'infrastructure	< 15 s	-	Faible (Calcul à l'instant T)
[17]	Méta-heuristique hybride (Vautour/Sauterelle)	Interférences, Énergie, QoS, Coût	Réduction de 72% à 86%	-	Faible (Réactif à la charge)
[2]	Heuristique d'orchestration (Règles explicites)	Équilibrage de charge, Latence URLLC	-	0.32 – 0.39 ms	Modérée (Lissage prédictif)
[8]	Algorithme Évolutionnaire	Débit, Latence	-	~ 0.30 ms (moyenne)	Forte (Insights via IA)

1.2.2 Apprentissage par renforcement

Le Reinforcement Learning (RL) offre une alternative aux approches précédentes en permettant à un agent d'apprendre une politique d'allocation par interaction directe avec l'environnement réseau, sans nécessiter de modèle analytique explicite. La littérature récente fait émerger trois problématiques distinctes : la gestion des contraintes sur de longs horizons, la sûreté des décisions en déploiement réel, et la coordination entre agents dans un contexte multi-tranches.

Cai et al. [4] adoptent le paradigme centralisé avec *PW-DRL* (Prediction-aided Weighted DRL). Le problème est formulé comme un processus décisionnel de Markov (MDP) à espace d'action hybride : les décisions d'admission des utilisateurs sont discrètes (accepter/rejeter chaque requête), tandis que l'allocation de puissance est continue. L'agent utilise l'algorithme TRPO (Trust Region Policy Optimization) pour apprendre la politique d'admission. TRPO contraint la mise à jour de la politique à rester dans une *région de confiance* définie par une divergence KL maximale entre l'ancienne et la nouvelle politique, garantissant ainsi une amélioration monotone et évitant les mises à jour catastrophiques. Pour la composante continue (allocation de puissance), les auteurs intègrent la théorie d'optimisation de Lyapunov plutôt que de confier cette tâche au DRL. Le principe est le suivant : pour chaque contrainte de long terme (puissance moyenne $\leq P_{max}$, capacité $\leq C_{max}$), une *file d'attente virtuelle* $Q(t)$ est définie dont la longueur évolue selon $Q(t+1) = \max[0, Q(t) + g(t) - c]$, où $g(t)$ est la consommation instantanée et c la contrainte. L'algorithme minimise alors la *drift-plus-penalty* de Lyapunov $\Delta L(t) + V \cdot f(t)$, où $\Delta L(t)$ est la variation de la fonction de Lyapunov quadratique $L(t) = \frac{1}{2} \sum_j Q_j^2(t)$, $f(t)$ est la fonction objectif instantanée, et $V > 0$ est un paramètre de compromis optimalité/stabilité. Cette décomposition transforme le problème contraint de long terme en une séquence de sous-problèmes non contraints par intervalle de temps, chacun soluble analytiquement. Une particularité supplémentaire est le *réseau de prédiction* : un module auxiliaire qui, à partir de l'état courant, prédit l'état futur de l'environnement et l'injecte dans l'observation de l'agent TRPO, améliorant significativement la qualité des décisions d'admission sous ressources limitées. Les résultats montrent que PW-DRL maintient la stabilité sur 300 intervalles de temps sans violation de contrainte, là où TRPO seul, SAC et les algorithmes génétiques saturent entre 50 et 150 intervalles. La politique apprise priorise intelligemment le trafic critique : plus de 80% d'admission pour les requêtes sensibles au délai (DS), contre environ 48% pour les requêtes gourmandes en débit (TO).

Toutefois, PW-DRL, comme la plupart des approches DRL, ne distingue pas les contraintes *cumulatives* (respecter un SLA moyen sur un long horizon) des contraintes *instantanées* (ne jamais dépasser un seuil de latence à un pas de temps donné). Or, en déploiement réel, un agent DRL peut prendre des décisions ponctuellement dangereuses tout en respectant la contrainte moyenne, ce qui est inacceptable pour des services immersifs à latence ultra-faible.

Nagib et al. [10] abordent directement cette lacune avec *SafeSlice*, une approche de DRL sûr (*safe DRL*) conçue pour le contexte O-RAN. Le problème est formulé comme un processus décisionnel de Markov contraint (CMDP), où l'agent doit minimiser la consommation de ressources (PRBs) tout en respectant deux types de contraintes de latence simultanément. Pour les contraintes *cumulatives*, les auteurs conçoivent une fonction de récompense multi-objectifs intégrant une composante sigmoïdale sensible au risque :

$$R = W_u \cdot \left(1 - \frac{\sum_s b_s}{B}\right) + W_l \cdot \sum_s W_s \cdot \frac{1}{1 + e^{c_{1,s} \cdot (l_s - c_{2,s})}} \quad (3)$$

où b_s est la bande passante consommée par la tranche s , B la bande totale, l_s la latence moyenne observée, et les paramètres c_1 (pente de la sigmoïde) et c_2 (point d'inflexion, calé sur le seuil SLA ϵ) contrôlent la sensibilité de la pénalité. Cette formulation impose un gradient de pénalité croissant à mesure que la latence s'approche du seuil, plutôt qu'une pénalité binaire franchissement/non-franchissement.

Pour les contraintes *instantanées*, SafeSlice introduit une *couche de sûreté* (*safety layer*) placée en sortie du réseau de neurones de l'agent. Cette couche intercepte chaque action a_t proposée par la politique et consulte un modèle de coût supervisé un régresseur XGBoost ($R^2 = 0.94$, RMSE = 4.42) entraîné sur des données état-action-coût collectées dans l'environnement O-RAN pour prédire la latence résultante $\hat{C}_i(\kappa_t, a_t)$. Si le coût prédit dépasse le seuil instantané ω_i pour une tranche quelconque, la couche de sûreté projette l'action sur l'action faisable la plus proche au sens euclidien :

$$a'_t = \arg \min_{a' \in \mathcal{A}_f} \|a' - a_t\|^2 \quad \text{avec } \mathcal{A}_f = \{a' \in \mathcal{A} \mid \hat{C}_i(\kappa_t, a') \leq \omega_i, \forall i\} \quad (4)$$

Cette projection garantit que l'action corrigée reste efficiente en termes de consommation de ressources tout en satisfaisant les seuils de latence.

L'évaluation est conduite dans un environnement O-RAN virtualisant trois services dont du trafic VR gaming réel, avec quatre catégories de tests combinant changements de trafic et de seuils SLA. SafeSlice réduit la latence cumulative moyenne de 83,23%, les violations instantanées de 93,24% et la consommation de ressources de 22,13% par rapport aux baselines (A2C non contraint, SAC-Lagrangian). Notamment, SAC-Lagrangian, pourtant fréquemment utilisé dans la littérature du slicing, échoue à adapter sa politique en ligne face à des changements de conditions réseau, ce qui confirme l'insuffisance des approches DRL contraintes classiques en contexte de déploiement dynamique. La principale limite de SafeSlice réside dans la qualité du modèle de coût : avec un RMSE de 4.42 sur une plage de latence $[0, 50]$ ms, la couche de sûreté peut laisser passer des violations lorsque le seuil est très restrictif (5 ms), soulignant la dépendance de l'approche à la précision du prédicteur de coût.

Les approches précédentes reposent sur un agent centralisé unique, ce qui pose deux problèmes : le passage à l'échelle lorsque le nombre de tranches augmente, et la coordination entre tranches concurrentes qui partagent un pool de ressources limité.

Abiko et al. [1] adressent le premier problème avec une architecture multi-agents basée sur Ape-X, une variante distribuée de DQN (Deep Q-Network). Les auteurs adoptent le paradigme « une tranche, un agent » : chaque agent Ape-X est un réseau de neurones profond indépendant qui observe l'état de sa propre tranche via un vecteur comprenant le taux de satisfaction QoS et le ratio d'utilisation des Resource Blocks (RB). L'espace d'action est discret et ternaire : $\{-1, 0, +1\}$, correspondant respectivement à une demande de diminution, de maintien ou d'augmentation du nombre de RB. La fonction de récompense est $r_t = \alpha \cdot \text{SLA_sat}(t) - \beta \cdot \text{waste}(t)$, pénalisant la sur-allocation. L'architecture Ape-X distribue l'entraînement : plusieurs *acteurs* (un par tranche) collectent des expériences en parallèle avec des politiques légèrement différentes (exploration ϵ -greedy avec des ϵ variés), tandis qu'un *learner* central agrège ces expériences dans un replay buffer priorisé et met à jour le réseau Q partagé. La coordination inter-agents reste cependant rudimentaire : un contrôleur réseau central agrège les demandes et arbitre la distribution des RB en priorisant les tranches les plus dégradées, sans véritable négociation entre agents. L'environnement est simulé via ns-3 avec des scénarios calibrés sur l'outil Ericsson Mobility Calculator. Les résultats montrent un taux de respect SLA d'environ 97% avec une sur-allocation minimisée, et une scalabilité native : l'ajout d'une nouvelle tranche ne nécessite que l'instanciation d'un nouvel agent sans ré-entraînement des existants.

Cependant, cette coordination par arbitrage central ne résout pas le problème fondamental des *conflicts inter-tranches* : lorsque plusieurs agents décident simultanément d'augmenter leur allocation sur la base de leur observation locale, la somme des demandes peut dépasser les ressources physiques disponibles. Rezazadeh et al. [12] abordent frontalement ce problème avec *STEP* (Standalone Explainable Protocol), un framework MADRL (Multi-Agent DRL) qui permet aux agents de développer un protocole de coordination *émergent* sans signalisation prédéfinie. Chaque agent STEP est associé à une tranche et prend deux types de décisions simultanées : une *action physique* (allocation de fréquences CPU dans le contexte O-RAN vRAN) et une *action de communication* (un message discret envoyé aux autres agents). Les messages n'ont aucune sémantique prédéfinie ; leur signification émerge au fil de l'entraînement par la pression de la fonction de récompense, qui pénalise les conflits de ressources et récompense la minimisation de la latence.

L'originalité architecturale de STEP réside dans l'intégration de la théorie de l'*Information Bottleneck* (IB) au sein du réseau de neurones de chaque agent. Une couche de goulot d'étranglement stochastique, inspirée des autoencodeurs variationnels (VAE), est insérée entre les couches d'entrée et de décision. Cette couche compresse l'observation (état local + messages reçus) en une représentation latente Z de dimension réduite (32 dans les expériences), en imposant une contrainte d'information mutuelle $I(X, Z) < I_c$ via un terme de divergence KL dans la fonction de perte. Le *reparameterization trick* ($Z = \mu + \sigma \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, 1)$) permet la rétropropagation à travers cette couche stochastique. Ce mécanisme a un double effet : il force les agents à ne retenir que l'information pertinente pour la décision (réduisant le surapprentissage à des états spécifiques), et il garantit que les messages échangés sont concis et sémantiquement riches plutôt que redondants.

Les expériences sont menées sur un environnement de slicing O-RAN à trois tranches (URLLC, eMBB, mMTC) avec des profils de trafic variables (eMBB : $23,33 \pm 22,38$ Mbps, URLLC : $7,80 \pm 9,25$ Mbps, mMTC : $14,80 \pm 20,86$ Mbps). STEP réduit les conflits inter-tranches d'un facteur $6,06\times$ par rapport à une approche par protocole prédéfini et de $3,4\times$ par rapport à un MADRL sans communication, atteignant zéro conflit en fin d'entraînement (500 épisodes). La latence médiane est ramenée à 0 ms (contre 4 ms pour le MADRL baseline), avec un IQR de 1,25 ms contre 11 ms, et la sous-utilisation des ressources CPU est réduite d'un facteur $1,4\times$. L'interprétabilité est analysée via les valeurs SHAP, qui révèlent que les messages inter-agents ont un impact plus fort sur les décisions d'allocation que les obser-

vations locales de trafic, confirmant l'émergence d'un véritable protocole de coordination. Toutefois, les auteurs identifient un point de saturation : au-delà de 8 symboles dans l'alphabet de communication, les gains marginaux disparaissent, suggérant que la complexité du protocole émergent reste intrinsèquement limitée par la dimension du problème.

L'analyse de ces travaux montrent que la question de la sûreté du DRL en déploiement réel, longtemps ignorée, commence à être adressée par [10], mais la distinction entre contraintes cumulatives et instantanées reste rare dans la littérature. Également, la coordination multi-agents évolue d'un simple arbitrage central [1] vers des protocoles émergents [12], mais la scalabilité de ces approches au-delà de trois tranches reste à démontrer. Enfin, aucun des travaux étudiés ne couple véritablement un modèle de prédiction du trafic à un moteur d'allocation dans une boucle fermée intégrée, les approches opèrent soit de manière réactive, soit avec une anticipation limitée.

TABLE 3 – Comparaison des modèles basés sur l'apprentissage par renforcement profond (DRL) pour l'allocation de ressources

Référence	Architecture DRL	Algorithmes & Mécanismes	Mécanisme de Sûreté / Contraintes	Espace d'action	Performances clés
[4] (PW-DRL)	Centralisée	TRPO + Optimisation de Lyapunov + Réseau de prédiction	Files d'attente virtuelles (gestion des contraintes long terme)	Hybride (Admission discrète, Puissance continue)	Stabilité sur 300 intervalles sans violation ; 80% d'admission pour trafic sensible (DS).
[10] (SafeSlice)	Centralisée (Safe DRL)	CMDP + Régresseur supervisé XGBoost (Modèle de coût)	Récompense sigmoïdale (cumulatif) + Couche de sûreté par projection (instantané)	Continu (corrigé par projection sur l'action faisable)	-93,24% de violations instantanées ; -83,23% de latence cumulative.
[1] (Ape-X)	Multi-Agents (MADRL)	Ape-X (DQN distribué avec replay buffer priorisé)	Pénalité de sur-allocation + Arbitrage central basique	Discret ternaire (-1, 0, +1 RB)	~ 97% respect SLA ; Scalabilité native (ajout de tranches sans ré-entraînement).
[12] (STEP)	Multi-Agents (MADRL coopératif)	DQN + Information Bottleneck (couche stochastique VAE)	Protocole de communication émergent pour éviter proactivement les conflits	Double : Action physique (CPU) + Action communication	Conflits réduits de $6,06 \times$ (zéro conflit final) ; Latence médiane de 0 ms.

1.3 Jeux de données

Le *network slicing* (5G et B5G) a démultiplié l'intérêt pour des approches data-driven. Dans ce contexte, les jeux de données disponibles publiquement se structurent principalement selon deux axes : des datasets **issus de simulation** et des datasets **issus de captures réelles**.

1.3.1 Données simulées

BNNetSimulator Le dataset BNNetSimulator [7] présente une grande variété en matière de données simulées. Cet ensemble contient 7900 scénarios de simulation générés via un simulateur basé sur OMNet++ et couvre 130 topologies différentes.

Chaque entrée du jeu de données se compose d'un ensemble (scénario, performances), ces mêmes entrées sont segmentées selon les trois types de tranches (eMBB, URLLC, mMTC). Le scénario inclut des informations à propos de la topologie réseau, les flux de trafic, les configurations de routage et les paramètres de découpage réseau. Les performances concernent la latence, la gigue ainsi que le taux de perte de paquets.

Bien que le simulateur utilisé travaille au niveau des paquets, les informations ainsi que les performances calculées ne sont pas propres aux paquets mais à un flux (ensemble des paquets envoyés d'une source à une destination pour une tranche spécifique). De plus, aucune information n'est fournie à propos des timestamps spécifiques aux paquets ou aux flux de données, seuls des KPIs temporels (délai et gigue) sont proposés.

RouteNet Ce dataset [3] a pour objectif d'entraîner des modèles de prédiction de performance, particulièrement des réseaux de neurones sur graphes (GNN), afin d'évaluer des politiques d'allocation de ressources. Il comprend plus de 480 000 entrées simulées sur trois topologies standards : NSFNet (14 nœuds), GEANT2 (24 nœuds) et une topologie synthétique de 50 nœuds.

Chaque échantillon est défini par un triplet : une topologie (G), une matrice de trafic (T) et une configuration de routage (R). Les caractéristiques d'entrée incluent la capacité des liens et l'intensité du trafic. En sortie, le dataset fournit des mesures moyennes de délai, de gigue et de taux de perte pour chaque chemin de bout en bout.

Il n'inclut pas nativement de segmentation par type de service ou par type de tranche, traitant tout le trafic de manière homogène, ce qui limite son application directe au partitionnement de ressources spécifiques par tranche (eMBB, URLLC, mMTC). Cette absence de timestamps limite également l'usage de modèles temporels.

1.3.2 Données réelles

CRAWDAD Le dataset CRAWDDAD UMKC Network Slicing 5G [14] est issu d'un banc d'essai utilisant des outils open-source (Open5GS, srsRAN) et des radios logicielles (USRP). Il simule un environnement 5G Standalone (SA) où le trafic est généré par des terminaux réels accédant à différents services.

Avec plus de 460 000 entrées, ce dataset est conçu pour entraîner des modèles de classification afin de prédire le slice (eMBB, URLLC, mMTC) en fonction des informations sur un paquet. Le jeu de données contient 117 caractéristiques (features) extraites au niveau du paquet et du flux. Parmi elles, on retrouve la taille des paquets (minimum, maximum, moyenne), les intervalles de temps entre paquets (Inter-Arrival Time – IAT), ainsi que des drapeaux TCP/IP spécifiques. Les labels sont explicitement définis selon les trois tranches standardisées : eMBB (flux vidéo), URLLC (jeux en ligne ou contrôle à distance) et mMTC (capteurs IoT). Bien qu'il inclue des informations temporelles, la précision de capture (par jour et par heure) reste assez large.

KimDaegyeom Ce dataset [6] représente des mesures directes de trafic 5G provenant d'un opérateur mobile majeur en Corée du Sud. Il capture les patterns réels de trafic 5G avec timestamps intégrés pour chaque enregistrement.

Ce dataset contient un total de 358 heures d'enregistrement d'envoi de paquets de données. Les données sont structurées autour de variables réseau telles que l'identifiant de la cellule, le type d'application (Streaming, Web, File Transfer) et des métriques de débit descendant/montant. Le point fort de ce dataset réside dans sa précision temporelle : chaque enregistrement est horodaté avec une précision allant jusqu'à la milliseconde, permettant une analyse très fine du trafic.

Cependant, la tranche réseau (eMBB, URLLC, mMTC) n'est pas explicitement étiquetée par l'opérateur pour des raisons de confidentialité. Il reste cependant possible d'effectuer un travail de rétro-ingénierie en corrélant le type de service avec les informations sur les paquets afin de recréer un étiquetage exploitable.

1.3.3 Synthèse

L'analyse comparative présentée dans le Tableau 4 révèle un compromis récurrent dans la littérature actuelle. D'un côté, les données simulées offrent un étiquetage précis des tranches réseau mais manquent de réalisme temporel. De l'autre, les données réelles offrent une précision à la milliseconde mais souffrent d'un manque de labels explicites dû aux contraintes de confidentialité des opérateurs. Il existe donc un besoin pour une approche capable de concilier la précision temporelle au niveau du paquet et la connaissance structurelle des tranches réseau, afin de permettre une gestion véritablement dynamique des ressources.

TABLE 4 – Comparaison synthétique des jeux de données pour le *network slicing*.

Dataset	Nature	Granularité	Timestamps	Précision	Slices	Objectif
BNNetSimulator	Simulation	Flux	Non	–	Oui	Performance KPI
RouteNet	Simulation	Flux	Non	–	Non	Optimisation GNN
CRAWDAD	Réel (Banc)	Paquet	Oui	heure	Oui	Classification
KimDaegyeom	Réel (Opérateur)	Paquet	Oui	ms	Non	Analyse Trafic

1.4 Synthèse générale

L'analyse croisée des trois parties de cet état de l'art - prédiction, allocation et jeux de données - fait émerger un constat : ces domaines sont étudiés de manière largement cloisonnée dans la littérature. Les modèles de prédiction (section 1.1) produisent des estimations de trafic sans se soucier de la manière dont elles seront consommées par un moteur d'allocation. Les modèles d'allocation (section 1.2) opèrent soit de manière réactive, soit avec une anticipation limitée à un lissage exponentiel, sans intégrer de véritable module prédictif en amont. Le seul travail qui tente cette intégration est [9], au prix d'une complexité de mise en œuvre qui freine son adoption opérationnelle. Parallèlement, les jeux de données disponibles (section 1.3) ne permettent pas d'évaluer une boucle fermée prédiction-allocation : les datasets simulés manquent de réalisme temporel, tandis que les données réelles manquent d'étiquetage par tranche.

Au-delà de ce cloisonnement, quatre verrous spécifiques restent ouverts. Tout d'abord, l'apprentissage fédéré pour la prédiction de trafic [11] n'a été évalué que sur des stations de profil homogène, sans jamais confronter le protocole FedAvg à l'hétérogénéité intrinsèque des différentes tranches (eMBB, URLLC, mMTC) au sein d'un réseau. Ensuite, l'intégration des contraintes SLA dans la fonction de perte, pourtant démontrée comme bénéfique par [16], reste cantonnée à une seule métrique (le trafic) sans extension à une loss multi-métrique différenciée par type de tranche. En troisième lieu, la plupart des évaluations reposent sur des données mono-site ou mono-scénario, ce qui limite la validité externe des conclusions. Finalement, aucun travail ne propose de pipeline couplant prédiction, détection de violation SLA et réallocation de ressources dans une boucle fermée opérationnelle.

Références

- [1] Yu Abiko, Takato Saito, Daizo Ikeda, Ken Ohta, Tadanori Mizuno, and Hiroshi Mineno. Flexible resource block allocation to multiple slices for radio access network slicing using deep reinforcement learning. *IEEE Access*, 8 :68183–68198, 2020.
- [2] Faezeh Bahramisirat, Mark A. Gregory, and Shuo Li. Dynamic network slicing for resource allocation in 5g/b5g networks : An optimization-based approach. *Ad Hoc Networks*, 186 :104194, 2026.
- [3] BNN-UPC. RouteNet-Erlang : A Graph Neural Network model for custom network topologies. <https://github.com/BNN-UPC/RouteNet-Erlang>, October 2021.
- [4] Yuekai Cai, Peng Cheng, Zhuo Chen, Wei Xiang, Branka Vucetic, and Yonghui Li. Dynamic resource allocation in network slicing with deep reinforcement learning. 2023.
- [5] Wei-Kun Chen, Ya-Feng Liu, Yu-Hong Dai, and Zhi-Quan Luo. Qos-aware and routing-flexible network slicing for service-oriented networks. *IEEE Transactions on Network and Service Management*, 22(6) :6021–6036, 2025.
- [6] Yong-Hoon Choi, Daegyeom Kim, and Myeongjin Ko. 5g traffic datasets, 2023.
- [7] J. P. Gómez-Álvarez, A. Aguilar-García, and J. F. Monserrat. Generation of a network slicing dataset : The foundations for ai-based b5g resource management. *Data in Brief*, 55 :110787, 2024.
- [8] Venkata Ramana K, Ramesh B, Ravindra Changala, Aditya Sai Srinivas T, Praveen Kumar Kalangi, and Bhavsingh M. Optimizing 6g network slicing with the evonetslice model for dynamic resource allocation and real-time qos management. *International Research Journal of Multidisciplinary Technology*, 6(3) :325–340, May 2024.
- [9] Ziran Min, Ziteng Liu, Zhuangwei Kang, Shuang Zhou, Swapna Gokhale, Shashank Shekhar, Charif Mahmoudi, and Aniruddha Gokhale. A novel 5g digital twin approach for traffic prediction and elastic network slice management. In *2024 16th International Conference on COMMunication Systems NETWORKS (COMSNETS)*, pages 497–505, 2024.
- [10] Ahmad M. Nagib, Hatem Abou-Zeid, and Hossam S. Hassanein. Safeslice : Enabling sla-compliant o-ran slicing via safe deep reinforcement learning. In *2025 IEEE International Conference on Machine Learning for Communication and Networking (ICMLCN)*, pages 1–7, 2025.
- [11] Nikolaos Pavlidis, Vasileios Perifanis, Selim F. Yilmaz, Francesc Wilhelmi, Marco Miozzo, Pavlos S. Efraimidis, Remous-Aris Koutsiamanis, Pavol Mulinka, and Paolo Dini. Federated learning in mobile networks : A comprehensive case study on traffic forecasting. *IEEE Transactions on Sustainable Computing*, 10(3) :576–587, 2025.
- [12] Farhad Rezazadeh, Hatim Chergui, Shuaib Siddiqui, Josep Mangués, Houbing Song, Walid Saad, and Mehdi Bennis. Intelligible protocol learning for resource allocation in 6g o-ran slicing. *IEEE Wireless Communications*, 31(5) :192–199, 2024.
- [13] Sajal Saha, Saikat Das, and Glaucio H.S. Carvalho. ConvLstmtransnet : A hybrid deep learning approach for internet traffic telemetry. In *2024 IEEE Virtual Conference on Communications (VCC)*, pages 1–6, 2024.
- [14] Anurag Thantharate, Cory Beard, Rahul Paropkari, and Vijay Walunj. Crawdad umkc/networkslicing5g, 2022.
- [15] Nguyen Phuc Tran, Oscar Delgado, Brigitte Jaumard, and Fadi Bishay. Ml kpi prediction in 5g and b5g networks. In *2023 Joint European Conference on Networks and Communications 6G Summit (EuCNC/6G Summit)*, pages 502–507, 2023.
- [16] Evren Tuna, Asude Baykal, and Alkan Soysal. Multivariate and multistep mobile traffic prediction with sla constraints : A comparative study. *Ad Hoc Networks*, 163 :103594, 2024.
- [17] Shabana Urooj, Rajesh Arunachalam, Mohamad A. Alawad, Kuldeep Narayan Tripathi, Damodaran Sukumaran, and Poonguzhali Ilango. An effective model for network selection and resource allocation in 5g heterogeneous network using hybrid heuristic-assisted multi-objective function. *Expert Systems with Applications*, 248 :123307, 2024.
- [18] Yang Yang, Yechen He, Binnan Zhao, Celimuge Wu, Zhipeng Gao, and Lanlan Rui. Multirepresentation spatial-temporal graph convolutional networks for network traffic prediction. *IEEE Internet of Things Journal*, 12(13) :23085–23099, 2025.